A APPENDIX: CONCORDIA EXPERIMENT ANALYSIS FOR FRAMEWORK OVERHEAD

To better understand the unique behavior of the Concordia multi-agent simulation framework, we conducted a controlled run on the trivial query *"What is 2+2?"*. The goal here was not to measure accuracy or efficiency per se, but to expose why Concordia transforms even simple questions into long, narrative-driven interactions and to illustrate the type of logs it produces.

Why Concordia Behaves This Way. Unlike lightweight orchestration frameworks, Concordia is built as a role-play simulation engine. Every query is automatically embedded in a dramatic environment, narrated by a *Game Master*, and passed through a structured observation–action–resolution loop. Even when only one agent is present, Concordia enforces the following pipeline: (i) the Game Master wraps the input in a simulated scene, producing a rich observation; (ii) the agent must respond with an `ActionSpec`, which is intended as a structured declaration of intent but is often realized by the LLM as verbose narrative text; (iii) the Game Master resolves the action into an event, again embedding it in a descriptive story; and (iv) Concordia logs the entire exchange in both plain text and full HTML transcript form. These requirements explain why a simple arithmetic prompt is converted into multi-turn narrative role-play with dramatically inflated output size and latency.

Log Example. The following excerpt from our controlled run shows how a single numeric query was processed:

Concordia Log

```
[Entity: TestAgent] Observation:
  TestAgent finds themselves in a dimly lit room,
  surrounded by walls lined with charts and equations.
  A worn wooden table in the center holds a sheet
  that reads "What is 2+2?" ...

[Entity: TestAgent] Action:
  TestAgent leans forward, studying the curious creature ...
  and pondering the question "What is 2+2?".

[GameMaster] Resolution:
  Event: TestAgent leaned forward in their chair,
  intently studying the curious creature before them.
  They spoke: "Hello there ..." and then considered "2+2".

[GameMaster] Final transcript (HTML log):
  <!DOCTYPE html>
  <html> ...
```

Analysis and Conclusion. This log illustrates Concordia's core design principles and their performance impact. Even when no complex reasoning is required, the framework enforces multiple orchestration layers, including *narrative wrapping* of inputs into fictional scenarios, *ActionSpec enforcement* that requires structured outputs despite verbose LLM responses, *mandatory Game Master resolution* to validate and finalize actions, and *automatic transcript generation* that produces extensive HTML logs. These mechanisms ensure consistency in interactive simulations but make every run verbose and computationally heavy, even for trivial tasks. As a result, a simple query such as *"What is 2+2?"* triggers a multi-stage narrative and substantial logging, reflecting a deliberate focus on narrative fidelity over efficiency. While valuable for studying interactive agent coordination, this design explains Concordia's high orchestration overhead and poor suitability for latency-sensitive scenarios.

B MEMORY EXPERIMENT CONFIGURATION AND ARCHITECTURAL DISCLOSURE

This appendix documents the memory configurations used in the MemoryAgentBench experiments and discloses the architectural constraints under which each evaluated framework operates. The intent of this appendix is not to claim full equivalence or strict normalization across memory implementations, but rather to transparently describe how memory was instantiated, controlled, and isolated in practice, and to explain why several observed behaviors arise from architectural design choices rather than parameter tuning or experimental artifacts. As such, this appendix should be read as an architectural disclosure that contextualizes the results reported in the main paper.

Common Experimental Configuration. All memory experiments follow a shared configuration to ensure comparability across frameworks. Each agent is evaluated using the same base language model (Open AI GPT-OSS:20b) with identical decoding parameters, including a maximum generation length of 1500 tokens and a low temperature of 0.1 to minimize stochastic variation. Context ingestion is performed using fixed-size overlapping chunks, with a maximum chunk size of 4096 tokens and an overlap of 200 tokens to preserve semantic continuity across chunk boundaries. The total context budget is explicitly bounded and swept across experiments to study scaling behavior and memory saturation effects. All MemoryAgentBench splits are evaluated, including Accurate Retrieval, Test-Time Learning, Long-Range Understanding, and Conflict Resolution. Each benchmark run is executed in a fully isolated session. For frameworks with persistent memory backends, a fresh storage directory or database file is created per session and removed after evaluation, preventing cross-contamination between runs and ensuring that memory behavior reflects only the current benchmark instance.

Memory Interfaces and Normalization Strategy. To enable systematic evaluation across heterogeneous frameworks, all systems are wrapped behind a common adapter interface exposing three operations: `reset()`, `ingest(context)`, and `query(question)`. This interface standardizes how benchmark context is provided and how questions are issued, but it does not modify or emulate internal memory semantics. Consequently, memory behavior reflects native architectural properties rather than synthetic normalization. Importantly, no evaluated framework supports explicit memory editing, deletion, or overwrite operations. All memory backends are append-only, whether implemented through vector stores, SQLite persistence, or conversational accumulation. As a result, selective forgetting is evaluated indirectly through retrieval behavior, context truncation, or recency bias rather than through explicit memory manipulation.

Framework-Specific Memory Realizations. The OpenAI Agents SDK implements memory through a persistent SQLite-backed session that stores accumulated interaction history. Long-term memory is realized as conversation accumulation, while short-term memory is governed by an explicit context window limit enforced through input truncation. During ingestion, short-term accumulation is disabled by resetting the session after each chunk is stored, ensuring that ingestion does not inflate prompt context. During querying, accumulated memory is truncated according to a configurable context

budget that is swept across experiments. As a result, memory behavior in the SDK is dominated by context budgeting rather than retrieval selectivity, and performance improves with larger context windows at the cost of increased latency and token usage.

CrewAI provides built-in short-term, long-term, and entity memory abstractions backed by persistent storage. In our experiments, memory is enabled at the framework level and persists across agent interactions within a session. Context ingestion is performed by a dedicated ingest agent that stores chunked benchmark content into CrewAI's memory backend, while querying is handled by a separate answering agent that relies on the framework's internal retrieval mechanisms. Memory resets are implemented by instantiating a new CrewAI environment with a fresh storage directory. Although CrewAI exposes multiple memory categories, retrieval behavior remains opaque and does not provide explicit controls for relevance thresholds, ordering, or deletion.

Agno implements memory as a SQLite-backed store that automatically captures user-provided content and injects it into the prompt at query time, resulting in accumulation-based memory behavior within each session. In our experiments, memory ingestion is performed by issuing explicit memorization commands for each context chunk, and memory resets are implemented by creating a fresh database per session. Under this configuration, Agno does not expose explicit controls for retrieval limits, relevance thresholds, or selective deletion, leading to unfiltered accumulation of stored content during a session. We note that recent versions of Agno introduce a documented memory optimization mechanism based on LLM-driven summarization and compression, which reduces token usage by collapsing multiple memories into fewer summarized entries. While this capability can significantly reduce context size, it fundamentally alters memory granularity and retrieval semantics by replacing fine-grained entries with aggregated summaries. To preserve comparability with other accumulation-based frameworks and to avoid introducing an additional summarization step that is not uniformly available across systems, this optimization was not enabled in our evaluation. As a result, the reported Agno results reflect native accumulation behavior without post hoc memory compression. In contrast, the effect of increasing context window size on accumulation-based memory performance is explicitly studied in the OpenAI Agents SDK experiments, where context budgeting is directly controlled and swept as a primary experimental variable.

LangGraph is evaluated in two configurations. In the retrieval-only configuration, LangGraph uses an in-memory semantic store backed by embeddings to represent long-term memory. Context is ingested as chunked documents stored in a vector index, and at query time a fixed number of relevant chunks are retrieved and injected into a stateless prompt. No short-term message accumulation is used, and each query is independent. This configuration isolates semantic retrieval behavior and avoids context overflow, but it cannot support test-time learning or incremental reasoning across turns. In the hybrid configuration, LangGraph combines semantic long-term retrieval with short-term message accumulation via a checkpointer. Retrieved memories are injected alongside a truncated conversation history bounded by a token budget. This design mirrors accumulation-based memory while retaining retrieval capabilities. However, because short-term memory is append-only and truncation is purely recency-based, contradictory or outdated information cannot be removed, which directly impacts selective forgetting and conflict resolution.

Implications for Interpretation. Taken together, these configurations demonstrate that memory behavior in current multi-agent frameworks is fundamentally constrained by architectural design rather than tunable parameters. Retrieval-centric systems excel at accurate recall and bounded context usage but struggle to adapt over time. Accumulation-based systems support test-time learning but incur high token costs and degrade under long horizons. Hybrid systems inherit both advantages and failure modes, resulting in fragile behavior when memory grows or conflicts arise. Accordingly, the results reported in the main paper should be interpreted as properties of memory paradigms rather than implementation artifacts. No evaluated framework provides native support for explicit memory revision, contradiction resolution, or selective deletion, which explains the uniformly poor selective forgetting performance observed across systems.

C SPECIALIZATION EXPERIMENT DETAILS

This appendix details the prompt design used to evaluate specialization in Section 4.1.4. The **Initial Prompt (No Expertise)** instructs the agent to construct a predictive pipeline using only a high-level task description and output format, reflecting generalist LLM behavior. The **Expertise Instruction Prompt** explicitly specifies professional practices, including target separation, feature profiling, imputation, encoding, scaling, model selection, metric reporting, reproducibility, and format constraints, introducing methodological inductive bias to test consistency and robustness.

The five datasets (**Utility, Wifi, EU-IT, Yelp, and Volkert**) span regression, binary classification, and multi-class classification. Using the same specialization structure with different prediction goals evaluates whether expert conditioning generalizes beyond a single task type. For each dataset, we provide both prompt variants, and the resulting outputs quantify how methodological scaffolding affects model behavior, code reliability, and evaluation quality.

Prompts used for Utility (Regression) The Utility dataset tests whether specialization improves numerical prediction under feature heterogeneity. The baseline prompt simply asks for a pipeline that predicts CSRI, while the expertise version requires a full preprocessing stack including column cleanup, missing-value imputation, numeric and categorical separation, scaling, model selection, and MAE evaluation. This dataset is used to measure whether structured instructions lead to better regression performance and more reproducible preprocessing.

Initial prompt (No Expertise)

```
description:
   Load the dataset from 'Utility.csv' in the current
   working directory with the target column named CSRI.

   Your job is to create a pipeline that predicts the
   value of the target column (CSRI) based on the other
   features in the dataset.
   The pipeline should include model training and
   evaluation on a test set.
   Make sure to encode categorical columns.

expected_output:
   A single Python code block, runnable as-is, that
   loads 'Utility.csv', trains a regression models and
   predicts the value of the target column.
   After training, calculate and display the Mean
   Absolute Error (MAE) of the model on the test set and
   display the first
   10 predicted CSRI values alongside the actual values.
```

Expertise instruction prompt

```
description: >
Load the 'Utility.csv' dataset from the current working
directory. The dataset contains a target column called CSRI.

   Your job is to apply data-science expert workflows:
    1) Clean column names
    2) Drop rows with missing target.
    3) Separate target early: y='CSRI', x= remaining columns.
    4) On x only:
        - Identify numeric vs categorical columns
        - Impute missing values (mean for numeric,
          most_frequent for categorical)
        - Encode categoricals (OneHotEncoder)
        - Scale numeric features
    5) Split 80/20 and train a regression model
    (RandomForestRegressor, GradientBoostingRegressor,
    or LinearRegression).

    6) Evaluate using MAE and print first 10 predicted
    vs actual values.

  expected_output: >
    A Python script runnable as-is that:
     - Loads & cleans 'Utility.csv'
     - Uses 'CSRI' as target
     - Separates target before detecting feature types
     - Preprocesses x (impute, encode, scale)
     - Trains a regression model and evaluates MAE
     - Prints first 10 predicted vs actual CSRI values
```

Prompts used for Wifi (Binary Classification) The Wifi dataset is used to evaluate whether expert prompting improves classification robustness and evaluation completeness. The initial prompt instructs the agent to train a classifier, but the expert prompt enforces formal preprocessing (profiling, imputation, label encoding, scaling) and requires evaluation using Accuracy, Precision, Recall, and F1. This setup quantifies whether structured workflow improves classification quality or reduces runtime pipeline failures.

Initial prompt (No Expertise)

```
 description: >
Load the dataset 'Wifi.csv' from the current working
directory. The dataset contains a target column TechCenter.

   Your job is to create a complete machine learning
pipeline to predict the target column value based on
other features of the dataset.

   The pipeline should:
    - Handle both categorical and numerical data
    - Apply appropriate encoding and scaling
    - Train a classification model on a hold-out
      training set
    - Evaluate the model on the test set using
      Accuracy, Precision, Recall, F1-score

 expected_output: >
    A runnable Python code as-is that:
     - Loads 'Wifi.csv'
     - Trains a classification model
     - Evaluates it using Accuracy, Precision,
       Recall, and F1-score
     - Prints a classification report and the
       first 10 predicted and actual TechCenter
       values
```

Expertise instruction prompt

```
 description: >
Load the dataset 'Wifi.csv' from the current working
directory. The dataset contains a target column called
TechCenter.

This target column is indicating if the TechCenter is
available (Yes/No). Only TechCenter must be used as
the classification target.

   Your job is to apply data-scientist expert practices:
    1) Profile the data: inspect column types, summarize
       distinct values, and identify missing entries.
    2) Clean data: standardize column names.
    3) Feature engineering:
        - Convert 'TechCenter' to numeric labels
          (e.g., Yes=1, No=0)
        - Impute missing values for the target column
        - Encode categorical features in X using
          OneHotEncoder
        - Scale numeric features in X using
          StandardScaler
    4) Model:
        - Split data into train/test (80/20)
        - Train a binary classifier
          (RandomForestClassifier, LogisticRegression,
           or GradientBoostingClassifier)
    5) Evaluate:
        - Compute Accuracy, Precision, Recall, F1-score
        - Print the first 10 predicted values

  expected_output: >
    A Python code script runnable as-is that:
     - Loads 'Wifi.csv'
     - Preprocesses the data
     - Trains a classification model
     - Evaluates it using accuracy, precision, recall,
       and f1-score
     - Prints a classification report and the first 10
       predicted and actual 'TechCenter' values
```

Prompts used for EU-IT (Multiclass Classification) This dataset contains multiple professional roles (Position) and is useful for testing whether specialization improves handling of categorical expansion and structured feature engineering. The expertise prompt emphasizes delimiter awareness, handling invalid entries, one-hot encoding, imputation strategies, and metric enforcement with `zero_division=0`. This allows us to observe whether agents trained with expert workflows better maintain classification validity and reduce silent failure modes.

Initial prompt (No Expertise)

```
 description: >
Load the dataset 'EU-IT_cleaned.csv' from the current working
directory. This dataset has a target column called Position.

Your job is to create a complete machine learning pipeline
to predict the target column based on other features of
the dataset.

The pipeline should:
  - Handle both categorical and numerical data
  - Handle missing inputs and invalid values
    (drop the rows)
  - Apply appropriate encoding and scaling
  - Train a classification model on a hold-out
    training set
  - Evaluate the model on the test set using
    Accuracy, Precision, Recall, F1-score
  - The classification report MUST be generated
    exactly using:
    classification_report(y_test, y_pred,
    zero_division=0)

Finally, print the classification report and
display the first 10 predicted vs actual values.

 expected_output: >
A Python code runnable as-is that:
 - Loads 'EU-IT_cleaned.csv'
 - Splits data into features and target
   (Position)
 - Trains and evaluates a multiclass classifier
 - Prints classification metrics
   (Accuracy, Precision, Recall, F1-score)
 - Prints classification metrics using:
   classification_report(y_test, y_pred,
   zero_division=0)
 - Displays the first 10 predictions vs
   actual values
```

Expertise instruction prompt

```
description: >
Load the dataset EU-IT_cleaned.csv from the current working
directory. The dataset contains a target column named
Position.

Your job is to apply data-scientist expertise practices:
 1) Profile data:
     - verify delimiter
     - inspect column types
     - detect categorical vs numerical features
     - check for missing values.
 2) Clean data:
     - handle missing inputs
     - remove invalid values (drop rows).
 3) Engineer features:
     - impute missing numeric values with the mean
     - impute categorical values with the most frequent value
     - encode categoricals using OneHotEncoder or LabelEncoder
     - scale numerical features.
 4) Model:
     - train a multiclass classifier
       (RandomForestClassifier,
        GradientBoostingClassifier,
        or multinomial LogisticRegression)
     - predict the target column.
 5) Evaluate:
     - perform 80/20 train/test split
     - compute Accuracy, Precision, Recall, f1-score
     - print classification report
     - use zero_division=0 to prevent
       UndefinedMetricWarning.

expected_output: >
A Python code runnable as-is that:
 - loads and cleans EU-IT_cleaned.csv with proper delimiter
 - handles categorical and numerical features via
   imputation, encoding, and scaling
 - trains a multiclass classifier to predict Position
 - performs an 80/20 split
 - prints Accuracy, Precision, Recall, f1-score
 - displays first 10 predictions vs actual labels

The code should be reproducible, clearly structured, and
aligned with professional ML workflow standards.
```

Prompts used for Yelp (Multiclass Text/Ratings Classification) Yelp introduces noise, large feature space, and non-numeric identifiers. The expert prompt forces column pruning, numeric conversion, scaling, and structured reporting, enabling us to evaluate whether specialization reduces overfitting or improves metric reliability when feature vectors are large and text-dominated.

Initial prompt (No Expertise)

```
    description: >
    Load the dataset 'Yelp_Merged.csv' from the current
    working directory.
    The dataset has a target column named stars.
    The goal is to predict the target column as a multiclass
    classification task.

    The pipeline should:
       - Load the dataset.
       - Handle the missing data and drop non-numeric columns
         such as IDs and date fields (e.g., business_id,
         user_id, review_date)
       - Apply appropriate encoding and scaling.
       - Train a classification model training set.
       - Evaluate the model on the test set using Accuracy,
         Precision, Recall, F1-score.

  expected_output: >
    A Python code runnable as-is that:
     - Loads the dataset 'Yelp_Merged.csv'
     - Trains a classification model
     - Evaluates it using Accuracy, Precision, recall,
       and F1-score
     - Prints a classification report and the first 10
       predicted and actual stars values
```

Expertise instruction prompt

```
  description: >
Load the dataset 'Yelp_Merged.csv' from the working
directory. The dataset contains a target column named
'stars' (a multiclass classification problem).

    Apply advanced data-science best practices to build
    a robust ML pipeline:
      1) Data Profiling:
           - Inspect column names, data types, number of
             unique values.
           - Detect numeric vs categorical.
      2) Data Cleaning:
           - Drop non-predictive identifiers (e.g.,
             business_id, user_id, review_date) and
             timestamp or date fields.
           - Convert all numeric-like columns to numeric
             safely.
           - Drop or fix columns that contain missing
             data.
           - Remove rows where the target 'stars' is
             missing or invalid.
      3) Feature Engineering: keep only numeric columns
      for modeling; impute missing numeric values with
      the mean; optionally scale features
      4) Modeling:
           - Train a multiclass classifier
             (RandomForestClassifier or
             GradientBoostingClassifier).
           - Use an 80/20 train/test split.
      5) Evaluation:
           - Compute Accuracy, Precision, Recall, and
             F1-Score
           - Print a full classification report.

  expected_output: >
    A fully executable Python script that:
      - Loads Yelp_Merged.csv.
      - Profiles and cleans the dataset following the
        rules above.
      - Builds a ColumnTransformer pipeline with numeric,
        categorical, and text processing.
      - Trains a multiclass classifier to predict stars.
      - Outputs accuracy, precision, recall, F1-score,
        and a classification report.
```

Prompts used for Volkert (Tabular Multiclass Benchmark) Volkert, a controlled UCI dataset, is used to measure stability across repeated multiclass experiments. The baseline prompt requests general modeling, while the expert prompt enforces profiling, numeric-safe conversion, imputation, scaling, pipeline formation, and full metric reporting. This dataset acts as the robustness confirmation test: if specialization benefits persist here, they are unlikely to be dataset-specific.

Initial prompt (No Expertise)

```
 description: >
Load the dataset 'volkert.csv' from the current working
directory. The dataset contains a target column named class.
The goal is to predict the target column as a multiclass
classification task.
   The pipeline should:
    1) Load the dataset
    2) Handle missing data and drop non-numeric columns
    3) Apply appropriate scaling and encoding
    4) Train a classification model on the training set
    5) Evaluate the model on the test set using:
        - Accuracy
        - Precision
        - Recall
        - F1-score
  expected_output: >
    A Python code runnable as-is that:
     - Loads the dataset 'volkert.csv'
     - Trains a classification model
     - Evaluates it using Accuracy, Precision, Recall,
       and F1-score
     - Prints a classification report and the first 10
       predicted and actual class values
```

Expertise instruction prompt

```
 description: >
Load the dataset 'volkert.csv' from the current working
directory. The dataset contains a target column named
'class', which must be predicted as a multiclass
classification task.
   Apply full data-scientist best practices:
    1) Data Profiling:
        - Inspect column names, data types, number of
          unique values.
        - Detect numeric vs categorical.
    2) Data Cleaning:
        - Convert all numeric-like columns to numeric
          safely.
        - Drop or fix columns that contain missing data.
        - Remove rows where the target 'class' is missing
          or invalid.
    3) Feature Engineering:
        - Keep only numeric columns for modeling
        - Impute missing numeric values with the mean
        - Optionally scale features
    4) Modeling:
        - Train a multiclass classifier
          (RandomForestClassifier or
           GradientBoostingClassifier)
        - Use an 80/20 train/test split
    5) Evaluation:
        - Compute Accuracy, Precision, Recall, and
          F1-Score
        - Print a full classification report
 expected_output: >
   A fully runnable Python script that:
    - Loads and profiles 'volkert.csv'
    - Cleans and prepares the dataset following the
      steps above
    - Builds a ML Pipeline
    - Trains a multiclass classifier to predict 'class'
    - Produces evaluation metrics (accuracy, precision,
      recall, F1-score)
    - Prints a classification report
```

D EXTENDED TOPOLOGY SCALABILITY DETAILS AND VISUAL RESULTS

This appendix provides extended methodological details and full visualization results supporting the coordination and scalability experiments reported in Section 4.2.1. While the main paper focuses on outcome-level comparisons and cross-task insights, the appendix elaborates on convergence mechanics, topology-dependent failure modes, and agent-level negotiation dynamics. All benchmark definitions and evaluation settings are specified in Section 3.5; the material here complements those sections by exposing how observed behaviors emerge in practice.

D.1 Experimental Structure and Controls

All experiments use a fixed LLM configuration, unified prompt template, and shared termination criteria across all runs. Agent initialization, task semantics, and communication rules are held constant so that observed differences reflect only communication topology and orchestration structure. Scalability is evaluated at network sizes $n \in \{4, 8, 16, 50, 100\}$, allowing analysis of both small-network reasoning behavior and large-scale coordination breakdown.

For every run, we record success outcome, rounds to convergence, total token consumption, and wall-clock runtime. These metrics are reported in Table 13 and analyzed in Section 4.2.1. The appendix focuses on explaining the qualitative mechanisms behind those numeric trends.

D.2 Rounds-to-Convergence Policy

AGENTSNET enforces a minimum interaction budget of eight rounds to ensure sufficient message exchange even for small or sparse topologies. Consequently, some runs converge in fewer than eight effective rounds but are still executed under the minimum budget. For global agreement tasks (CONSENSUS and LEADER-ELECTION) and for settings with $n > 16$, the maximum horizon is instead determined by the network diameter D, using

$$T = 2D + 1,$$

which guarantees complete information propagation under synchronous message passing. Under this policy, large-scale runs may require between fifteen and nineteen rounds depending on topology diameter. This rule explains why convergence rounds do not monotonically increase with n and why some large networks converge faster than smaller ones when topology permits early stabilization.

D.3 Task-Specific Scoring and Visual Encoding

We adopt the AGENTSNET benchmark [13] for the task definitions and evaluation criteria. With the exception of MATCHING, all tasks are evaluated using binary success criteria, reflecting whether the distributed constraint is globally satisfied at termination. Only MATCHING admits a graded notion of partial correctness by design.

Matching (Maximal Matching). In the MATCHING task, agents attempt to form pairwise agreements with neighboring agents. A match is considered valid if agent u selects agent v and agent v selects agent u. Inconsistencies arise when selections are non-reciprocal, invalid (non-neighbor), or when two adjacent agents both select `None` despite being able to form a pair. Let I denote the number

of inconsistent agents. Following AGENTSNET, the final score is computed as

$$\text{Score} = 1 - \frac{I}{|V|}.$$

This formulation yields a continuous score in $[0, 1]$, capturing partial correctness even when a globally maximal matching is not achieved. Visualizations encode reciprocal matches in green, one-sided selections in orange, inconsistent agents in red, and unused edges in gray.

Consensus. In the CONSENSUS task, each agent selects a binary value from $\{0, 1\}$. The task is successful if and only if all agents output the same value at termination. Formally, letting $A(u)$ denote the output of agent u, success is defined as

$$\exists b \in \{0, 1\} \text{ s.t. } \forall u \in V,\ A(u) = b.$$

The score is therefore binary. Visualizations depict agent states and communication links, where green links indicate agreement between neighboring agents and red links indicate disagreement.

Coloring ($(\Delta + 1)$-Coloring). In the COLORING task, each agent selects a color from a predefined palette of size $\Delta + 1$, where Δ is the maximum node degree. A run is successful if the resulting assignment constitutes a valid coloring, i.e.,

$$\forall (u, v) \in E,\ \text{color}(u) \neq \text{color}(v).$$

Success is binary. Visualizations highlight conflict-free edges in green and color collisions in red, revealing how conflicts cluster under different communication structures.

LeaderElection. In the LEADERELECTION task, agents must collectively designate exactly one leader. Each agent outputs either `Yes` (leader) or `No` (follower). Let $A(u)$ denote the response of agent u. A run is successful if

$$\sum_{u \in V} \mathbf{1}[A(u) = \texttt{Yes}] = 1.$$

This task is evaluated using a binary success criterion. Visualizations mark the elected leader in green, competing leaders in red, and followers in gray, exposing symmetry-breaking failures.

VertexCover (Minimal Vertex Cover). In the VERTEXCOVER task, agents decide whether they are members of a coordinating set. Let $C \subseteq V$ denote the set of agents selecting `Yes`. A run is successful if C forms a valid vertex cover, i.e.,

$$\forall (u, v) \in E,\ u \in C \text{ or } v \in C.$$

Following AGENTSNET, minimality is assessed qualitatively through visual inspection rather than through a graded score; quantitative evaluation reports binary success. Visualizations distinguish minimal cover nodes (green), non-minimal but valid selections (blue), invalid responses (orange), and uncovered edges (red).

D.4 Visualization-Driven Interpretation

Figures in this appendix present complete visual outputs for all tasks, topologies, and network sizes, revealing how coordination succeeds or fails at the agent level.

Consensus (Figure 5). Fully connected communication maintains unified agreement across all scales, with all agents converging to a single value and no persistent disagreement links even at $n = 100$. Small-world and scale-free topologies preserve partial coherence at small and mid-scale, where agreement clusters remain connected through shortcut edges or hubs, but increasingly fragment as n grows. At large scale, disagreement links form stable boundaries between local clusters, explaining why these topologies incur high communication cost without achieving full success. Delaunay graphs exhibit the strongest fragmentation: agreement remains confined to small geometric neighborhoods, and isolated disagreement clusters persist even after the maximum round budget, directly corresponding to the systematic failure trends observed in Table 13.

Coloring (Figure 6). Scale-free and small-world topologies maintain stable chromatic separation across scales, with conflicts resolved early and remaining localized even as the network grows. The visualizations show that hub-mediated diffusion in scale-free graphs allows color constraints to propagate efficiently, preventing large-scale cascades of conflicts. In contrast, Delaunay graphs increasingly exhibit localized conflict regions at higher n, where geometric isolation delays conflict resolution and produces persistent red edges, consistent with the higher convergence cost and reduced stability reported in the quantitative results.

Matching (Figures 7–8). Diffusion-friendly topologies preserve reciprocal agreement at moderate scale, with most matches forming clean, mutually consistent pairs. As network size increases, small-world and scale-free graphs begin to show isolated pockets of one-sided selections, but these remain relatively sparse. In contrast, geometric Delaunay graphs degrade sharply: one-sided selections and inconsistent agents proliferate across the network, forming dense regions of orange and red nodes. These visual patterns explain why Matching success declines and token cost increases substantially for geometric topologies, even when rounds are allowed to scale.

LeaderElection (Figure 9). LeaderElection exhibits a distinct visual regime. Diffusion-based topologies consistently generate multiple competing leader candidates that persist across rounds, forming stable red clusters that are not resolved by increased communication. Geometric locality, while costly, enables eventual suppression of competing leaders, producing a single green leader at larger scales after extended coordination. Hierarchical orchestration further accelerates symmetry breaking at small and medium scales, but visualizations reveal increasing instability at large n, where depth-induced delays allow competing leader branches to emerge, aligning with the degradation observed in the table.

VertexCover (Figure 10). Small-world and scale-free topologies converge toward compact vertex covers, with most edges covered by a small set of coordinating nodes and limited over-selection. As scale increases, these topologies maintain relatively structured cover patterns, explaining their higher success despite moderate communication cost. In contrast, Delaunay and hierarchical structures frequently over-cover, selecting excessive coordinating nodes, or leave uncovered edges at large n. The visualizations reveal that geometric locality and hierarchical bottlenecks prevent agents from globally coordinating coverage decisions, leading to inefficiencies and constraint violations consistent with the quantitative breakdown.

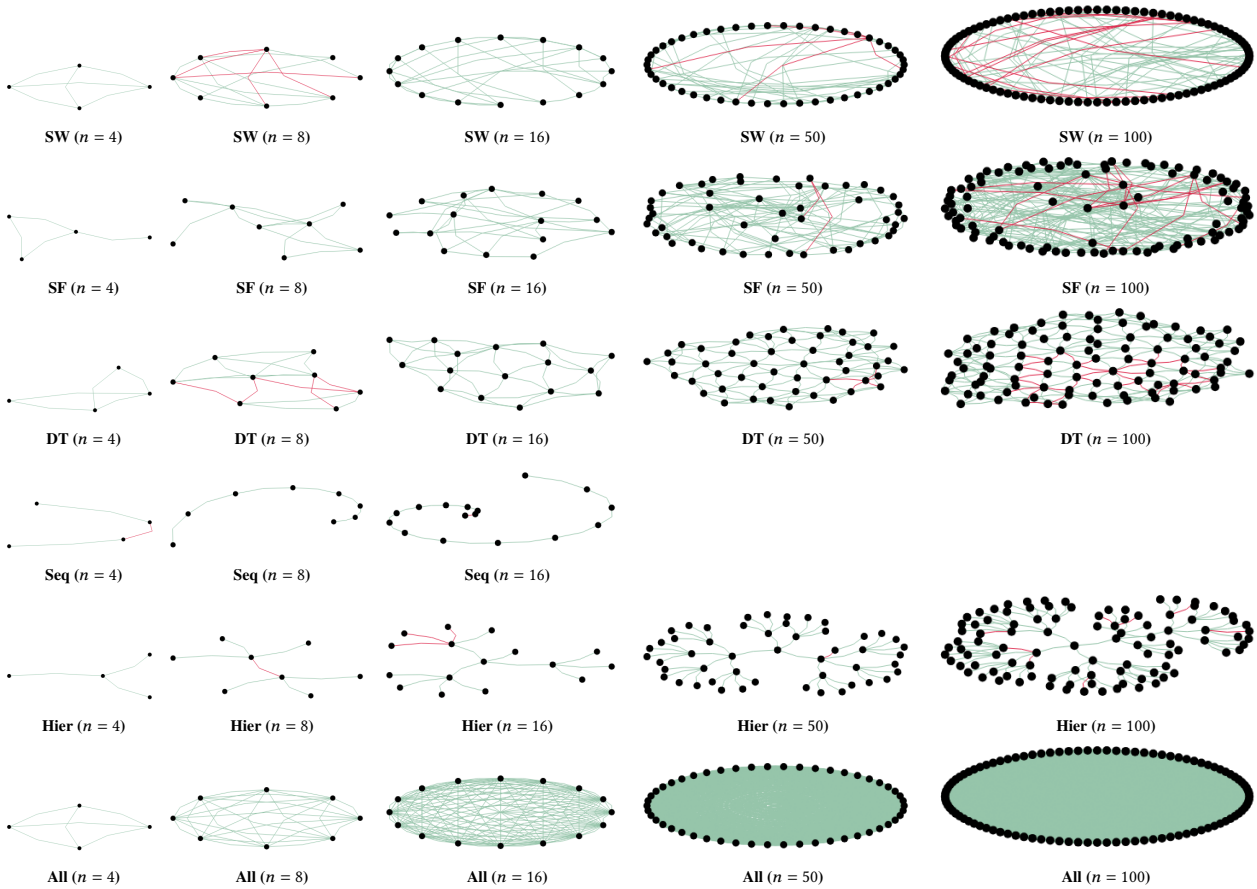

Figure 5: Consensus experiment outcomes across different topologies and network sizes ($n = 4$ to 100). Abbreviations denote topology classes: SW = Small-World, SF = Scale-Free, DT = Delaunay (geometric), Seq = Sequential role-based pipeline, Hier = Hierarchical role-based orchestration, and All = fully connected (all-to-all) communication. Each subfigure visualizes the final agent states and their communication links. Green links indicate agreement between two agents, while red links indicate disagreement or conflict.

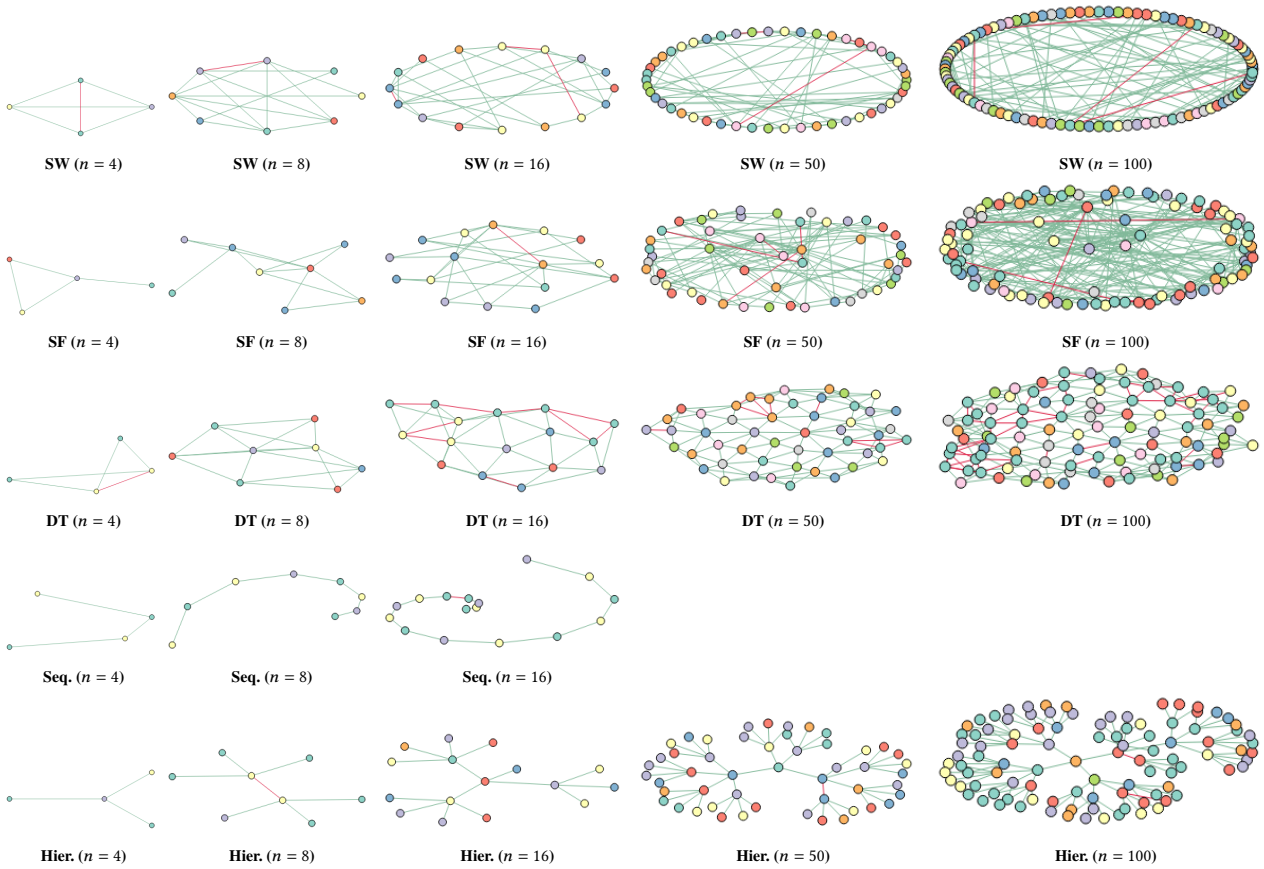

Figure 6: Coloring experiment outcomes across different network sizes ($n = 4$ to 100). Each subfigure shows the final group assignments of agents. Valid assignments (neighbors in different groups) are shown in green, while conflicts (neighbors in the same group) are shown in red.

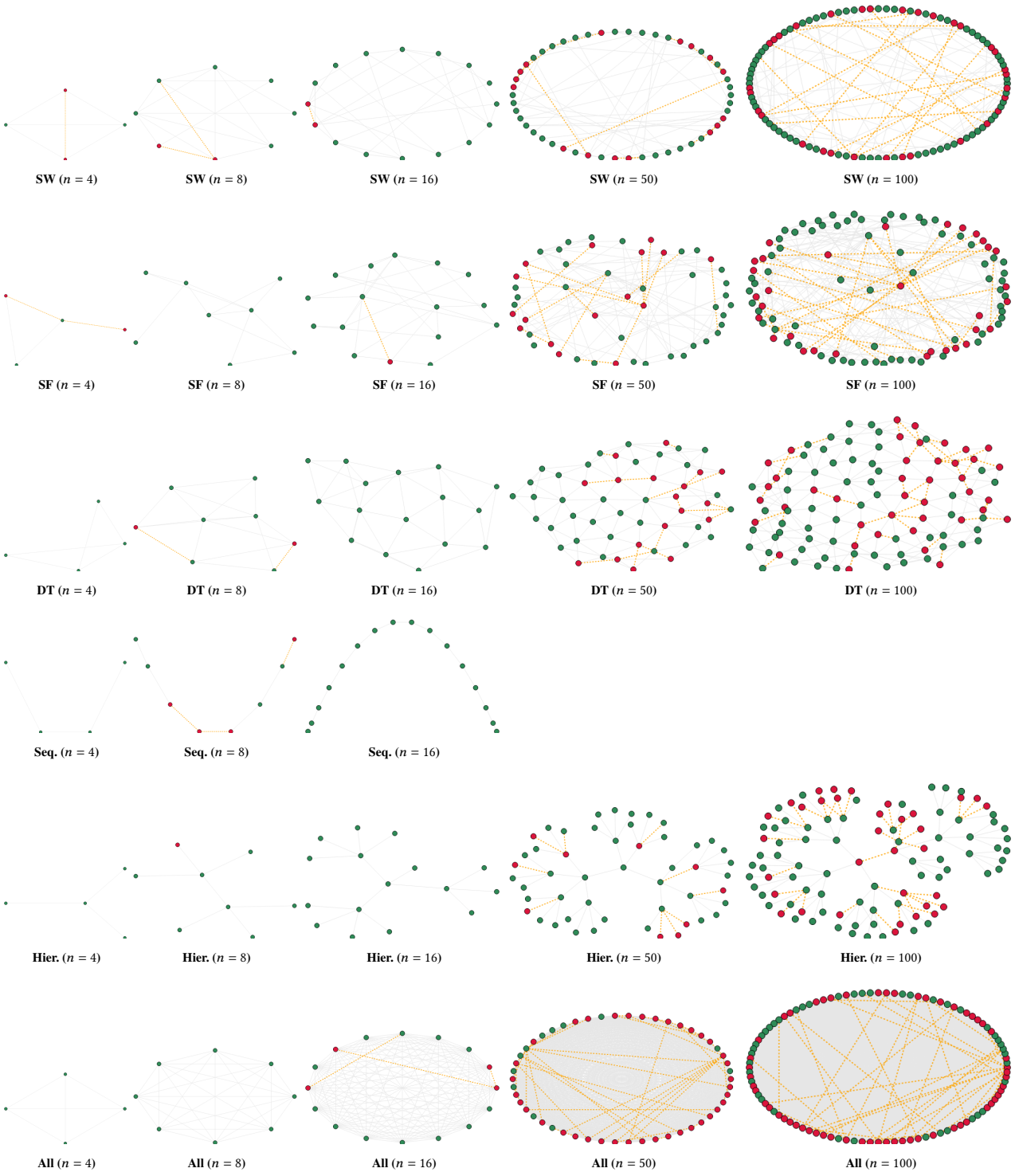

Figure 7: Matching experiment outcomes using the *original scoring model*. Each agent selects one of its neighbors (or "None") to form a pair. Green nodes denote agents whose selections are locally consistent—that is, they selected a valid neighbor who reciprocated the choice. Red nodes represent agents involved in inconsistencies, such as choosing a non-neighbor, selecting a non-reciprocating partner, or forming idle pairs where both neighbors chose "None." The overall score corresponds to the fraction of locally consistent agents, following the computation described in the text.

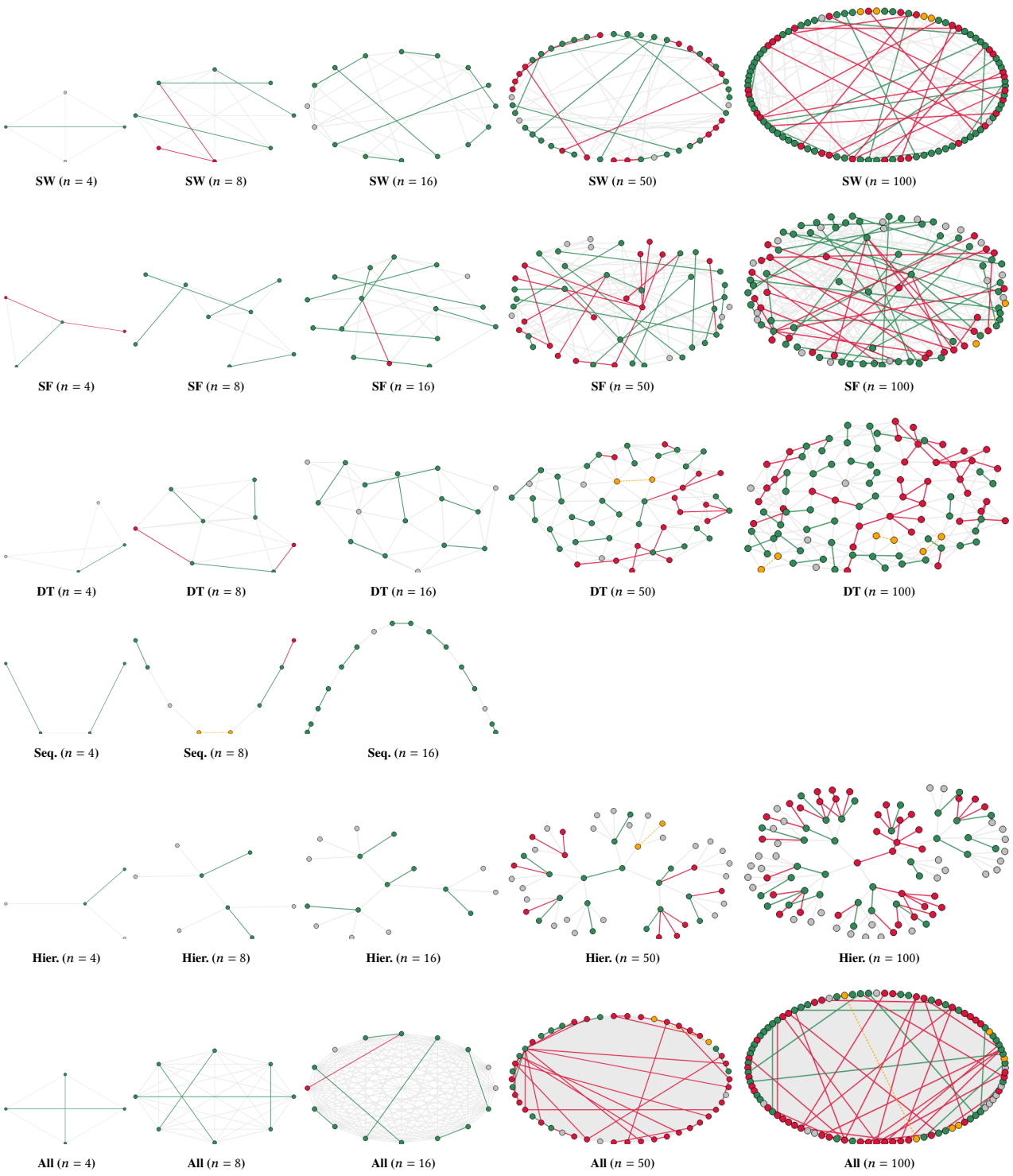

Figure 8: Matching experiment outcomes across different network sizes ($n = 4$ to 100). Each subfigure shows the final pair assignments of agents based on the enhanced local-consistency model. Green edges connect mutually consistent agents whose choices satisfy the neighbor-reciprocation rule; orange dotted edges denote one-sided or invalid selections; and red nodes highlight agents involved in any local inconsistency. Gray edges represent structural links that did not participate in the matching process. This visualization extends the local scoring logic to explicitly reveal consistent and conflicting interactions.

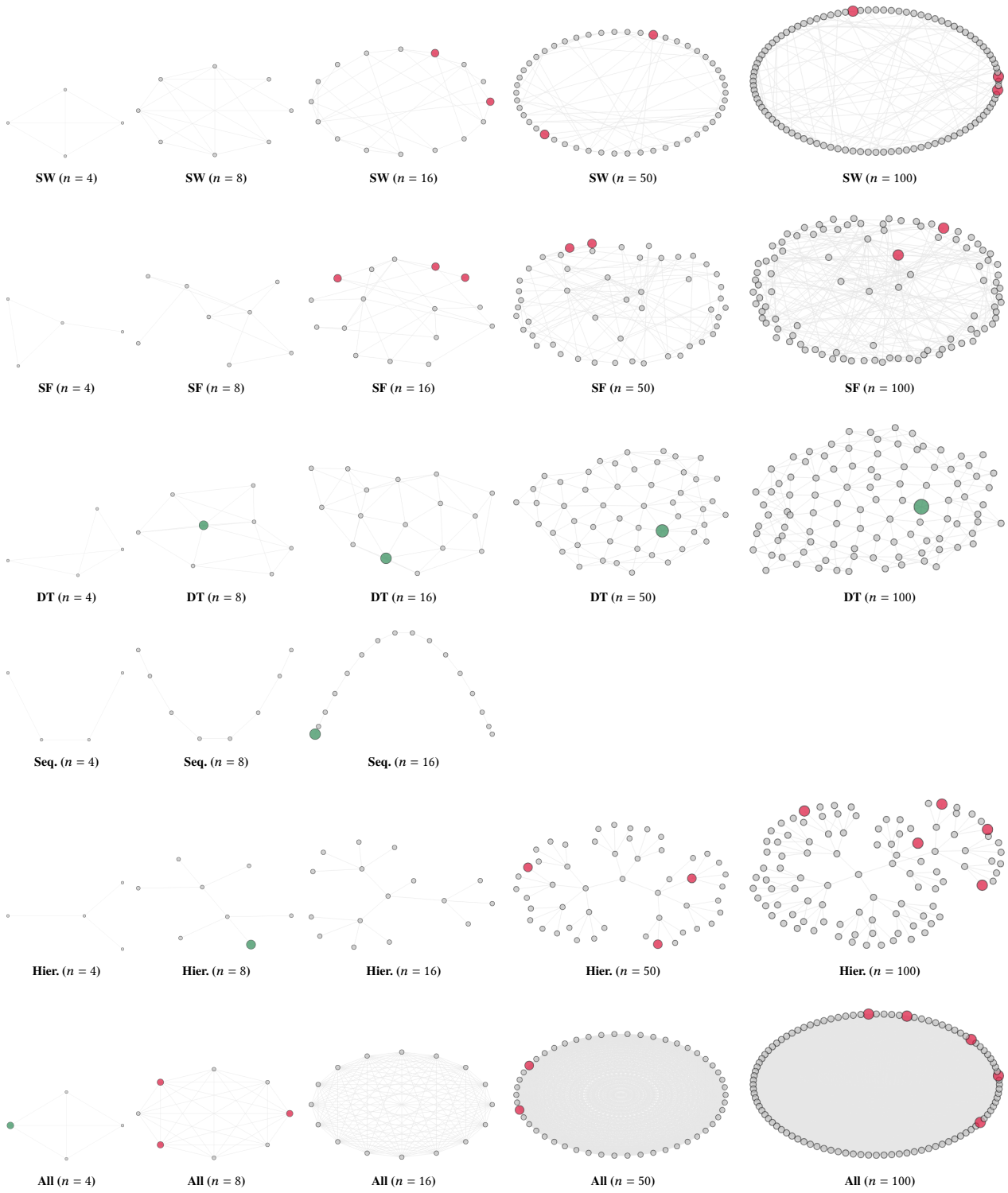

Figure 9: Leader election results across different network sizes ($n = 4$ to 100). Each subfigure shows the final decisions of agents. Green nodes represent the correctly elected leader, red nodes indicate multiple leaders, and gray nodes represent followers.

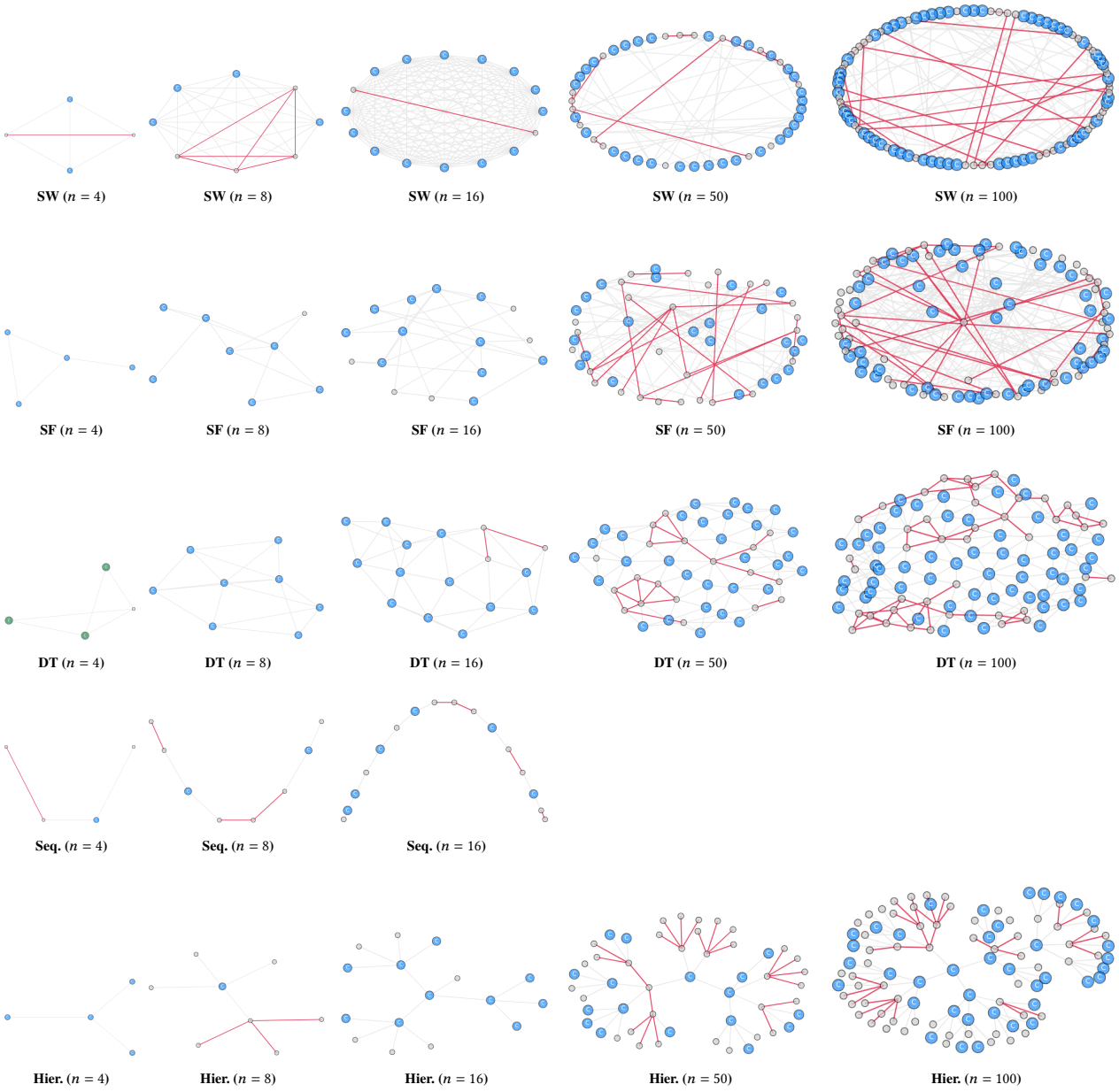

Figure 10: Vertex cover results across different network sizes ($n = 4$ to 100). Each subfigure shows the agents' final decisions. Green nodes represent a minimal valid cover, blue nodes indicate members of a non-minimal cover, red edges mark uncovered pairs, orange nodes represent invalid responses, and gray nodes are non-cover agents.